

## Analyse numérique et calcul scientifique - 1

### TP 3 (3h) Intégration Numérique

## 1 Objectifs

- Programmer en réutilisant des fonctions déjà développées en Python (création d'une librairie).
- Implementation de famille des méthodes Newton-Cotes et Quadrature Gauss-Legendre.
- Etudier les notions de précision et convergence pour ces méthodes.

## 2 Quadrature de Gauss-Legendre

1. En appliquant la procédure d'orthonormalisation de Gram-Schmidt (voir annexe des notes pour rappel) à la base canonique des polynômes  $\{1, x, x^2, x^3\}$ , et au produit scalaire  $(f, g) = \int_{-1}^1 f(x)g(x) dx$ , montrer que la quadrature de Gauss-Legendre à 3 points sur l'intervalle  $[-1, 1]$  a pour nœuds  $\xi_i \in \{-\sqrt{3/5}, 0, \sqrt{3/5}\}$  et pour poids  $w_i \in \{5/9, 8/9, 5/9\}$ .<sup>1</sup>
2. Quel est le lien entre les polynômes obtenus à l'item précédent et les polynômes de Legendre définis par (similitudes et différences)

$$L_M(x) = \frac{1}{2^M M!} \frac{d^M}{dx^M} (x^2 - 1)^M, \quad M \in \{0, 1, 2, 3\}. \quad (1)$$

3. Implémenter la fonction `GLquadrature` qui retourne  $\int_a^b f(x) dx$ , avec  $f : [a, b] \rightarrow \mathbb{R}$ , et qui reçoit comme paramètres :
  - **a** : limite gauche de l'intervalle d'intégration.
  - **b** : limite droite de l'intervalle d'intégration.
  - **f** : fonction à intégrer de type  $f : [a, b] \rightarrow \mathbb{R}$ .
  - **n** : le nombre de points d'intégration.

La fonction doit fonctionner pour  $n \in \{1, 2, 3\}$ . N'oubliez pas que les nœuds sont définis dans l'intervalle  $[-1, 1]$ , il faut faire le changement de variable correspondant.

4. Tester la fonction implémentée contre les résultats obtenus par `scipy.integrate.fixed_quad(f, a, b, n)[0]`.

## 3 Méthode de Newton-cotes

1. Implémenter la méthode de Newton-Cotes (NC) pour 2 (trapèze), 3 (Simpson), 4 et 5 points. Comme rappel, la méthode de  $NC_n$  à  $n$  points s'écrit:

$$\int_a^b f(x) dx \approx \mathcal{I}^{NC_n}(f) = (b - a) \sum_{i=1}^n B_{i,n} f(x_i), \quad (2)$$

---

<sup>1</sup>Au lieu d'effectuer toutes les intégrales à la main, il vous est permis de les calculer numériquement par n'importe quel schéma qui intègre exactement ces intégrales. C'est à vous de juger si cela est plus simple.

avec  $x_i = a + (i - 1) \times (b - a) / (n - 1)$ , pour  $i = 1, \dots, n$ . Les poids  $B_{i,n}$  sont données en appendix. La fonction implementé sera nommé `newton_cotes` ayant comme arguments les mêmes que `GLquadrature`.

2. Tester la fonction implementé contre les résultats obtenues par `GLquadrature` (choisissez des cas où les quadratures sont exactes).

## 4 Degré d'exactitude et ordre de convergence

1. D'abord on va transférer les fonctions `GLquadrature` et `newton_cotes` pour le fichier `quadrature.lib.py` afin de créer une librairie d'intégration réutilisable par plusieurs scripts. Il suffit d'ajouter `from quadrature.lib import *` en début du script lors que vous voulez utiliser cette librairie.
2. Exécuter le script `degree_exactitude.py` pour produire les tracées que mettre en évidence les degrés d'exatitute pour les méthodes implementés. Si vous avez bien suivi les conventions de noms, ce script devra exécuter sans aucune changement. Les degrés d'exactitude correspondent-ils aux valeurs théoriques attendues?
3. Ensuite on veut tester l'ordre de convergence de ces schémas en ce que l'intégration composée par morceaux. On se propose de calculer l'intégrale dans le temps du signal mesuré dans le fichier `Data0.txt`. Pour simplifier les calculs, au lieu de considérer les données brutes, on utilisera plutôt une représentation analytique de celles-ci. En utilisant une méthode d'interpolation par fonctions exponentielles, on parvient à l'expression suivante:

$$f(x) = c_0 + c_1 \exp(-x/T_1) + c_2 \exp(-x/T_2), \quad (3)$$

où  $c_0 = 1.05$ ,  $c_1 = 1$ ,  $c_2 = 1$ ,  $T_1 = 1$  et  $T_2 = 100$ . Le script `main_morceaux.py` peut être utilisé comme guide, ainsi comme `degree_exactitude.py`. Vous devez montrer la convergence des 8 méthodes implementés, dans une même tracée erreur versus pas de discretisation  $H$  en échelle logarithmique sur les deux axes ( astuce : raffinez  $H$  comme suit  $H_k = (b - a)/2^k$ ,  $k = 1, \dots, 13$ ).

4. Calculer les ordres de convergence numériques avec les deux derniers valeurs d'erreurs (astuce : vous devez calculer  $p$  en supposant  $E_k = CH_k^p$ ,  $E_{k+1} = CH_{k+1}^p$ ).

## 5 Approximation de fonctions

1. Dédurre les matrices du système linéaire résultant du problème de moindres carrés généralisé ci-dessous :

$$\min_{p \in V^n} \frac{1}{2} \|p - f\|^2, \quad (4)$$

où  $f : [-1, 1] \rightarrow \mathbb{R}$  est une fonction donnée,  $\|\star\| = \sqrt{\int_{-1}^1 (\star(x))^2 dx}$ , et  $V^n = \text{span}\{\phi_0, \phi_1, \dots, \phi_n\}$  est un espace polynomial de degré  $n$ , tel que  $p(x) = \sum_{i=0}^n a_i \phi_i(x)$ , où les  $a_i$  sont les coefficients à déterminer.

2. Quel est le lien entre ces matrices et le système linéaire associé à la matrice de Vandermonde du TP2 ?
3. Quelle base adoptée pour  $V^n$  rendrait les calculs plus simples ?
4. Choisir la même fonction génératrice que dans l'exercice 2.4 (TP2) dans le cadre de la régression et comparer les résultats obtenus par cette nouvelle méthode.

## Annexes

### A Paramètres des formules de quadrature

#### Newton-Cotes

| $n$ | $B_{1,n}$ | $B_{2,n}$ | $B_{3,n}$ | $B_{4,n}$ | $B_{5,n}$ |
|-----|-----------|-----------|-----------|-----------|-----------|
| 2   | 1/2       | 1/2       |           |           |           |
| 3   | 1/6       | 4/6       | 1/6       |           |           |
| 4   | 1/8       | 3/8       | 3/8       | 1/8       |           |
| 5   | 7/90      | 32/90     | 12/90     | 32/90     | 7/90      |

#### Gauss

| $n_p$ | $w_i$         | $\xi_i$                      |
|-------|---------------|------------------------------|
| 1     | 2             | 0                            |
| 2     | 1, 1          | $-1/\sqrt{3}, 1/\sqrt{3}$    |
| 3     | 5/9, 8/9, 5/9 | $-\sqrt{3/5}, 0, \sqrt{3/5}$ |